GUIDE PRATIQUE ILLUSTRÉ

Pipeline de données de ventes

Snowflake • dbt • Airflow • Kafka

Installer, configurer et reproduire le projet sur Windows, pas à pas.

Configuration par clé RSA • commandes prêtes à copier
Captures réelles • dépannage

Zouhair Serrar

ENSA Berrechid - Systèmes d'Information et Big Data

GUIDE DE REPRODUCTION / 02

Comment utiliser ce guide

Ce document permet à une autre personne de reproduire le lancement du projet fourni dans l’archive **dbt-snowflake-airflow-pipeline-main(2).zip**. Il décrit un environnement local de démonstration, connecté à Snowflake.

Parcours conseillé

Étape	Pages
Comprendre le projet et préparer Windows	3 à 5
Créer l’environnement et les données Snowflake	6 à 10
Configurer l’identité et l’authentification par clé	11 à 14
Lancer Docker, dbt et Airflow	15 à 19
Démarrer Kafka et alimenter Snowflake	20 à 23
Vérifier les résultats et refaire la démonstration	24 à 25
Arrêter, redémarrer et dépanner	26 à 28
Connaître les limites et compléter les preuves	29 à 30

Conventions de lecture

- **SQL / Snowflake** : exécuter dans l’éditeur SQL du navigateur.
- **PowerShell / Windows** : exécuter dans un terminal Windows, depuis le dossier indiqué.
- **Code / fichier** : modifier le fichier mentionné, puis enregistrer.
- Les valeurs **VOTRE_...** sont à remplacer. Ne pas réutiliser les identifiants d’une autre personne.

Pour des commandes PowerShell sur plusieurs lignes, le caractère accent grave en fin de ligne sert à poursuivre la commande. Copier le bloc entier, sans ajouter d’espace après ce caractère.

Périmètre de la validation

La démonstration réalisée le 17 octobre 2026 a confirmé une connexion dbt par clé, quatre tâches Airflow réussies, vingt commandes Kafka chargées et une table finale de vingt et une commandes. Les montants Kafka sont aléatoires et peuvent différer lors d’une nouvelle exécution.

GUIDE DE REPRODUCTION / 03

Comprendre le pipeline



Airflow orchestre dbt ; Docker héberge Airflow et Kafka.

Composant	Fonction dans ce projet
Producteur Python	Simule une boutique et produit des commandes JSON.
Kafka	Conserve et transporte les messages du topic orders.
Consommateur Python	Charge les commandes et leurs lignes dans Snowflake.
Snowflake	Stocke les tables et exécute les requêtes SQL générées par dbt.
dbt Core	Construit quatre vues de staging, une table résultat et lance les tests.
Airflow	Orchestre les quatre tâches dbt et planifie leur exécution.
Docker Desktop	Exécute les conteneurs Airflow et Kafka sous Windows.

> **Comprendre la fréquence**
> Le consommateur charge un micro-lot après 50 commandes ou environ 30 secondes à partir du premier message du lot. La table finale est reconstruite au lancement du DAG, planifié chaque jour à 00 :00 UTC. L’ensemble du pipeline n’est donc pas actualisé en continu.

Deux fichiers Compose sont utilisés : celui de la racine pour Airflow, et celui de kafka-lab pour Kafka. Toujours vérifier le dossier courant avant de lancer docker compose.

GUIDE DE REPRODUCTION / 04

Préparer Windows

Installer les outils suivants avant de copier les commandes. Le navigateur et les scripts Python restent sur Windows ; Airflow et Kafka tournent dans des conteneurs Linux.

Outil	Préparation
Docker Desktop	Installer la version Windows, activer le moteur WSL 2 et la virtualisation si l'installation le demande. Démarrer Docker Desktop avant le projet. [R1]
Python	Installer Python 3.12 pour le parcours proposé et le lanceur py. Les conteneurs de la validation utilisent Python 3.12.6.
Éditeur	VS Code ou le Bloc-notes pour éditer YAML et Python.
Snowflake	Disposer d'un compte, d'un accès à l'éditeur SQL et des droits de création nécessaires.
Ressources	Prévoir environ 8 Go de RAM disponibles pour une démonstration confortable et plusieurs Go libres. Ces valeurs sont indicatives.

PowerShell

```
docker --version
docker compose version
py --version
```

Si docker ne communique pas avec le moteur, ouvrir Docker Desktop et attendre son démarrage. Si py est introuvable mais python fonctionne, remplacer py par python dans les commandes Windows.

Port local	Utilisation
8081	Interface Airflow : http ://localhost :8081
8085	Interface Kafka UI : http ://localhost :8085
9092	Connexion Kafka des scripts Python Windows

GUIDE DE REPRODUCTION / 05

Extraire le projet au bon endroit

Décompresser l'archive. Pour simplifier les chemins de ce guide, créer **C :\Projets\pipeline-ventes** et y placer le contenu du dossier qui contient directement Dockerfile et dbt_project.yml.

Structure attendue

```
C:\Projets\pipeline-ventes\
    Dockerfile
    docker-compose.yaml
    dbt_project.yml
    profiles.example.yml
    dags\dbt_dag.py
    models\staging\
    models\marts\
    tests\snowflake_test.yml
    kafka-lab\
        docker-compose.yaml
        producer.py
        consumer_snowflake.py
        requirements.txt
```

Ouvrir PowerShell puis définir le chemin du projet. Si le dossier est ailleurs, remplacer uniquement la valeur de $Projet. Cette variable est à redéfinir dans chaque nouveau terminal.

PowerShell

```
$Projet = "C:\Projets\pipeline-ventes"
Set-Location $Projet
Get-Item .\Dockerfile, .\dbt_project.yml
Get-Item .\docker-compose.yaml
Test-Path .\profiles.example.yml
```

Point de contrôle
Le dernier résultat doit être True. Si PowerShell cherche profiles.example.yml dans C :\Users\..., le terminal n'est pas dans le dossier du projet. Utiliser Set-Location $Projet avant la copie.

Le ZIP fourni ne contient pas les CSV historiques. Les pages 8 à 10 créent un jeu minimal reproductible. Les chiffres anciens du README (300 historiques, 79 Kafka, etc.) ne décrivent pas ce jeu de démonstration.

GUIDE DE REPRODUCTION / 06

Accéder à l’éditeur SQL Snowflake

1. Ouvrir app.snowflake.com et se connecter. Si aucun compte n’existe, utiliser le parcours d’essai Snowflake classique sur signup.snowflake.com. Les conditions affichées peuvent évoluer.

2. Dans le menu, ouvrir **Projects** → **Workspaces**, puis **My Workspace** → **Add new** → **SQL File**. Si l’interface propose encore Worksheets, créer une feuille SQL. [R2]

3. Nommer le fichier **preparation_finance.sql**. Sélectionner un rôle autorisé. Le parcours validé utilise ACCOUNTADMIN sur un compte personnel de démonstration.

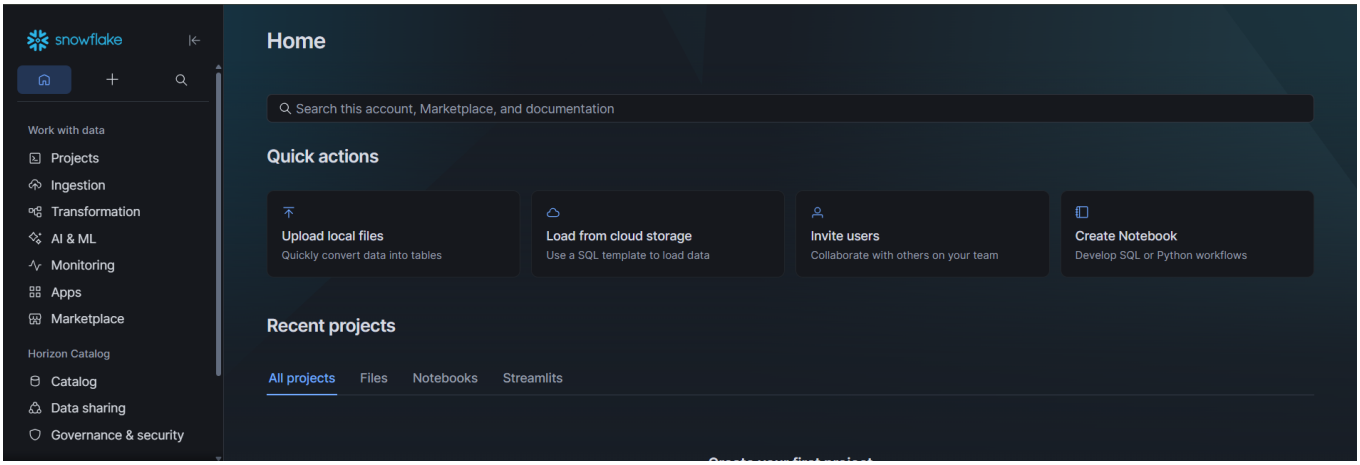


Figure 01. Accueil Snowflake : accès à Projects. Capture recadrée pour conserver la zone de navigation utile.

> **À retenir**
> Le parcours « Get started with CoCo » n’est pas nécessaire au projet. Le pipeline utilise le calcul SQL Snowflake et une connexion par clé ; il ne dépend pas de l’assistant CoCo.

Pour exécuter un bloc : le sélectionner puis cliquer sur le triangle bleu. Pour exécuter tout le fichier dans l’ordre : utiliser Run All. Un clic simple peut ne lancer que la requête sous le curseur.

GUIDE DE REPRODUCTION / 07

Créer le calcul et l’espace de données

Dans Snowflake, exécuter ce bloc complet. Le warehouse fournit le calcul, la base regroupe les données et le schéma RAW contient les objets du projet.

SQL

```
USE ROLE ACCOUNTADMIN;

CREATE WAREHOUSE IF NOT EXISTS FINANCE_WH
WITH
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;

CREATE DATABASE IF NOT EXISTS FINANCE_DB;
CREATE SCHEMA IF NOT EXISTS FINANCE_DB.RAW;

USE WAREHOUSE FINANCE_WH;
USE DATABASE FINANCE_DB;
USE SCHEMA RAW;

SELECT CURRENT_WAREHOUSE() AS WAREHOUSE_ACTUEL,
       CURRENT_DATABASE() AS BASE_ACTUELLE,
       CURRENT_SCHEMA() AS SCHEMA_ACTUEL;
```

WAREHOUSE_ACTUEL	BASE_ACTUELLE	SCHEMA_ACTUEL
FINANCE_WH	FINANCE_DB	RAW

XSMALL convient au petit volume présenté. AUTO_SUSPEND = 60 suspend le calcul après une période d’inactivité ; AUTO_RESUME permet de le redémarrer lorsqu’une requête l’utilise. La création avec IF NOT EXISTS conserve les objets existants.

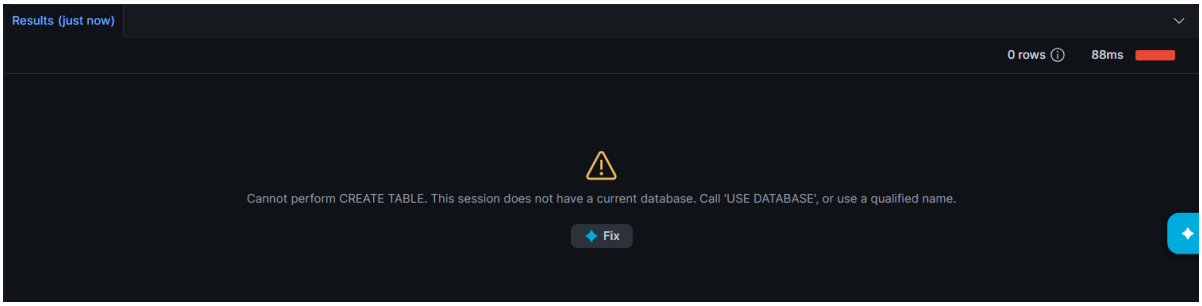


Figure 02. Erreur réellement rencontrée lorsque CREATE TABLE est lancé sans base courante. Exécuter les instructions USE avant la création.

GUIDE DE REPRODUCTION / 08

Créer les tables historiques

Les modèles du projet attendent exactement les noms de colonnes ci-dessous. Exécuter dans la même session Snowflake ; les noms complets rendent la destination explicite.

SQL

```
CREATE TABLE IF NOT EXISTS FINANCE_DB.RAW.CUSTOMERS (
    ID NUMBER,
    NAME VARCHAR,
    EMAIL VARCHAR,
    COUNTRY VARCHAR
);

CREATE TABLE IF NOT EXISTS FINANCE_DB.RAW.PRODUCTS (
    ID NUMBER,
    NAME VARCHAR,
    CATEGORY VARCHAR,
    PRICE NUMBER(12, 2)
);

CREATE TABLE IF NOT EXISTS FINANCE_DB.RAW.ORDERS (
    ID NUMBER,
    CUSTOMER_ID NUMBER,
    ORDER_DATE DATE,
    STATUS VARCHAR
);

CREATE TABLE IF NOT EXISTS FINANCE_DB.RAW.ORDER_ITEMS (
    ID NUMBER,
    ORDER_ID NUMBER,
    PRODUCT_ID NUMBER,
    QUANTITY NUMBER,
    UNIT_PRICE NUMBER(12, 2)
);
```

Relation	Signification
ORDERS.CUSTOMER_ID → CUSTOMERS.ID	Le client associé à la commande.
ORDER_ITEMS.ORDER_ID → ORDERS.ID	Les lignes appartenant à une commande.
ORDER_ITEMS.PRODUCT_ID → PRODUCTS.ID	Le produit présent sur chaque ligne.

Si une table existe déjà, IF NOT EXISTS ne corrige pas son schéma. Vérifier par DESCRIBE TABLE FINANCE_DB.RAW.ORDERS ; avant de réutiliser une ancienne base. Ces relations logiques ne sont pas vérifiées par les deux tests dbt actuels.

GUIDE DE REPRODUCTION / 09

Créer les tables de streaming

Ces tables sont obligatoires dès le premier lancement de dbt : les modèles effectuent une union entre l’historique et le flux, même si Kafka n’a encore rien envoyé.

SQL

```
CREATE TABLE IF NOT EXISTS FINANCE_DB.RAW.ORDERS_STREAM (
    ID NUMBER,
    CUSTOMER_ID NUMBER,
    ORDER_DATE DATE,
    TOTAL_AMOUNT NUMBER,
    STATUS TEXT,
    KAFKA_PARTITION NUMBER,
    KAFKA_OFFSET NUMBER,
    INGESTED_AT TIMESTAMP_NTZ
);

CREATE TABLE IF NOT EXISTS FINANCE_DB.RAW.ORDER_ITEMS_STREAM (
    ID NUMBER,
    ORDER_ID NUMBER,
    PRODUCT_ID NUMBER,
    QUANTITY NUMBER,
    UNIT_PRICE NUMBER,
    INGESTED_AT TIMESTAMP_NTZ
);

SHOW TABLES IN SCHEMA FINANCE_DB.RAW;
```

Colonne	Utilisation
KAFKA_PARTITION / KAFKA_OFFSET	Repèrent le message lu dans Kafka.
INGESTED_AT	Date technique du chargement ; utilisée par dbt pour dédupliquer le flux.
TOTAL_AMOUNT	Montant porté par le message ; le modèle résultat recalcule le montant à partir des lignes.

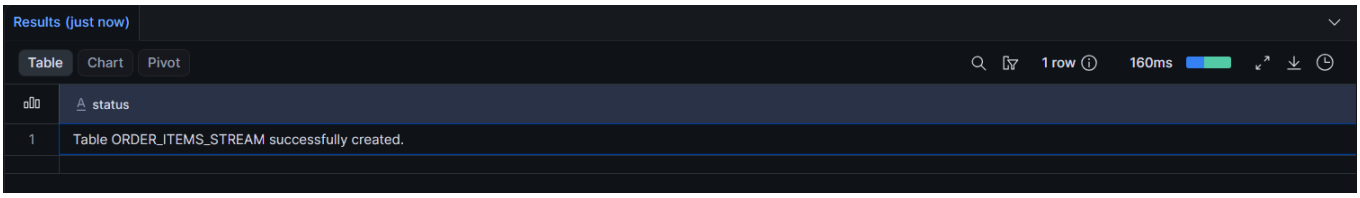


Figure 03. Confirmation observée : ORDER_ITEMS_STREAM a été créée.

> **À retenir**
> ORDERS_STREAM et ORDER_ITEMS_STREAM sont des tables ordinaires. Le suffixe STREAM n’implique pas un objet Snowflake CREATE STREAM.

GUIDE DE REPRODUCTION / 10

Charger le jeu de démonstration

Utiliser ce jeu uniquement dans une base de démonstration neuve. Il crée une commande de deux unités à 100, soit **200**. Les clauses NOT EXISTS évitent une réinsertion de l’identifiant 1 ; elles ne remplacent pas d’anciennes lignes.

SQL

```
INSERT INTO FINANCE_DB.RAW.CUSTOMERS
    (ID, NAME, EMAIL, COUNTRY)
SELECT 1, 'Client Demo', 'demo@example.com', 'Maroc'
WHERE NOT EXISTS (
    SELECT 1 FROM FINANCE_DB.RAW.CUSTOMERS WHERE ID = 1
);

INSERT INTO FINANCE_DB.RAW.PRODUCTS
    (ID, NAME, CATEGORY, PRICE)
SELECT 1, 'Produit Demo', 'Demo', 100
WHERE NOT EXISTS (
    SELECT 1 FROM FINANCE_DB.RAW.PRODUCTS WHERE ID = 1
);

INSERT INTO FINANCE_DB.RAW.ORDERS
    (ID, CUSTOMER_ID, ORDER_DATE, STATUS)
SELECT 1, 1, CURRENT_DATE(), 'Completed'
WHERE NOT EXISTS (
    SELECT 1 FROM FINANCE_DB.RAW.ORDERS WHERE ID = 1
);

INSERT INTO FINANCE_DB.RAW.ORDER_ITEMS
    (ID, ORDER_ID, PRODUCT_ID, QUANTITY, UNIT_PRICE)
SELECT 1, 1, 1, 2, 100
WHERE NOT EXISTS (
    SELECT 1 FROM FINANCE_DB.RAW.ORDER_ITEMS WHERE ID = 1
);
```

Table	Lignes attendues
CUSTOMERS / PRODUCTS / ORDERS / ORDER_ITEMS	1 dans chacune des quatre tables
ORDERS_STREAM / ORDER_ITEMS_STREAM	0 avant le premier chargement Kafka

Cohérence des références
Le producteur original tire des clients entre 1 et 100 et des produits entre 1 et 10. Pour reproduire exactement la démonstration avec ce jeu minimal, la page 22 restreint ces deux tirages à l’identifiant 1. Les montants restent aléatoires.

GUIDE DE REPRODUCTION / 11

Trouver les bons identifiants

Dans Snowflake, exécuter la première requête pour connaître l'organisation, le compte et le nom de l'objet utilisateur. Ces informations ne contiennent pas de mot de passe.

SQL

```
SELECT CURRENT_ORGANIZATION_NAME() AS ORGANISATION,
       CURRENT_ACCOUNT_NAME() AS COMPTE,
       CURRENT_USER() AS NOM_OBJET_UTILISATEUR;
```

Construire **account = ORGANISATION-COMPTE**, avec un tiret, sans https :// ni suffixe .snowflakecomputing.com. Exemple fictif : ORGA123-COMPTE456.

Le nom de l'objet utilisateur peut différer du nom de connexion. Remplacer la valeur de SHOW USERS par le NOM_OBJET_UTILISATEUR retourné ci-dessus, puis exécuter les deux instructions successivement.

SQL

```
SHOW USERS LIKE 'VOTRE_NOM_OBJET_UTILISATEUR';

SELECT "name", "login_name", "has_password",
       "must_change_password"
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

Information	Où la réutiliser
name / CURRENT_USER()	Nom de l'objet visé par ALTER USER pour associer la clé.
login_name	Valeur du champ user dans profiles.yml.
has_password	Indique seulement si un mot de passe est défini. La connexion par clé n'en a pas besoin.

A has_password	A must_change_password
true	false

Figure 04. État de l'utilisateur observé, recadré sans son identité. Un mot de passe défini ne garantit pas une connexion sans MFA.

> **À retenir**
> Dans la session validée, le premier refus venait d'un login_name différent du nom d'objet. Après correction, Snowflake a demandé un code TOTP. Le parcours suivant utilise une clé pour les connexions automatisées.

GUIDE DE REPRODUCTION / 12

Générer la paire de clés sur Windows

La clé privée reste dans le dossier personnel .dbt. La clé publique sera associée à l’utilisateur Snowflake. Le script réutilise la clé locale si elle existe déjà. [R3]

PowerShell

```
New-Item -ItemType Directory -Force "$env:USERPROFILE\.dbt"
py -m pip install cryptography
$env:SF_USER_OBJECT = "VOTRE_NOM_OBJET_UTILISATEUR"
```

Remplacer VOTRE_NOM_OBJET_UTILISATEUR par la valeur name de la page 11. Puis copier le bloc entier ci-dessous dans PowerShell.

PowerShell contenant du Python

```
@'
import os
from pathlib import Path
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

folder = Path.home() / ".dbt"
path = folder / "snowflake_key.p8"
if path.exists():
    key = serialization.load_pem_private_key(
        path.read_bytes(), password=None)
else:
    key = rsa.generate_private_key(
        public_exponent=65537, key_size=2048)
    path.write_bytes(key.private_bytes(
        serialization.Encoding.PEM,
        serialization.PrivateFormat.PKCS8,
        serialization.NoEncryption()))
public = key.public_key().public_bytes(
    serialization.Encoding.PEM,
    serialization.PublicFormat.SubjectPublicKeyInfo
).decode()
(folder / "snowflake_key.pub").write_text(public)
public = "".join(public.splitlines()[1:-1])
user = os.environ["SF_USER_OBJECT"].replace('"', '""')
sql = 'ALTER USER ' + chr(34) + user + chr(34)
print(sql + " SET RSA_PUBLIC_KEY = '" + public + "';")
'@ | py -
```

Cette variante de démonstration écrit une clé privée PKCS#8 sans phrase secrète. Protéger son accès et ne pas la transmettre. Ne pas déposer .dbt ou la clé privée dans un dépôt partagé.

GUIDE DE REPRODUCTION / 13

Associer la clé et créer le profil dbt

1. PowerShell affiche une commande ALTER USER contenant la clé publique. Copier la commande complète, puis l’exécuter dans Snowflake avec un rôle autorisé à gérer les clés de cet utilisateur.

SQL

```
-- Forme illustrative : utiliser la vraie commande générée.
ALTER USER "VOTRE_NOM_OBJET_UTILISATEUR"
SET RSA_PUBLIC_KEY = 'CLE_PUBLIQUE_GENEREE';
```

2. Dans PowerShell, depuis la racine du projet, créer la copie du profil. Si le fichier existe déjà, le sauvegarder et conserver ses autres profils avant modification.

PowerShell

```
Set-Location $Projet
Copy-Item .\profiles.example.yml `
    "$env:USERPROFILE\.dbt\profiles.yml"
notepad "$env:USERPROFILE\.dbt\profiles.yml"
```

3. Remplacer le profil mon_projet par ce bloc et renseigner uniquement account et user avec les valeurs de la page 11. Supprimer la ligne password de ce profil. [R4]

Fichier Windows : %USERPROFILE%\.dbt\profiles.yml

```
mon_projet:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "VOTRE_ORGANISATION-VOTRE_COMPTE"
      user: "VOTRE_LOGIN_NAME"
      private_key_path: "/home/airflow/.dbt/snowflake_key.p8"
      role: "ACCOUNTADMIN"
      warehouse: "FINANCE_WH"
      database: "FINANCE_DB"
      schema: "RAW"
      threads: 4
```

Enregistrer avec Ctrl + S. Le nom doit être profiles.yml, et non profiles.yml.txt. L’indentation YAML utilise des espaces. Le chemin de clé ci-dessus est volontairement le chemin Linux du conteneur.

GUIDE DE REPRODUCTION / 14

Vérifier les fichiers et les chemins

Le même dossier .dbt est lu depuis Windows et depuis Docker. Il n’est pas nécessaire de copier la clé privée dans le dossier du projet.

Usage	Chemin
Profil sur Windows	%USERPROFILE%\.dbt\profiles.yml
Clé privée sur Windows	%USERPROFILE%\.dbt\snowflake_key.p8
Profil dans Docker	/home/airflow/.dbt/profiles.yml
Clé dans Docker	/home/airflow/.dbt/snowflake_key.p8
Projet dans Docker	/opt/airflow/mon_projet

PowerShell

```
Test-Path "$env:USERPROFILE\.dbt\profiles.yml"
Test-Path "$env:USERPROFILE\.dbt\snowflake_key.p8"
```

Les deux réponses doivent être True. Dans docker-compose.yaml, vérifier la présence du montage suivant sous volumes :

Extrait YAML existant

```
- ${USERPROFILE}/.dbt:/home/airflow/.dbt:ro
```

Le suffixe ro indique une lecture seule dans le conteneur. Une modification du profil sur Windows sera relue au prochain lancement dbt ; elle ne nécessite pas de reconstruction de l’image.

> **Deux environnements, deux chemins**
> Le consommateur Kafka s’exécute sur Windows. Sa fonction de connexion utilisera donc ~/.dbt/snowflake_key.p8, et non le chemin /home/airflow/... du profil Docker. Cette adaptation est détaillée page 21.

GUIDE DE REPRODUCTION / 15

Construire et démarrer Airflow

Ouvrir Docker Desktop. Dans un terminal PowerShell, à la racine du projet, lancer :

PowerShell

```
$Projet = "C:\Projets\pipeline-ventes"
Set-Location $Projet
docker compose up -d --build
docker compose ps -a
```

Le premier démarrage télécharge l’image Airflow et installe dbt dans un environnement Python séparé. Attendre le retour de l’invite PowerShell.

Service	État attendu	Interprétation
airflow-init	Exited (0)	Initialisation terminée ; ce service est conçu pour s’arrêter.
airflow-webserver	Up, puis healthy	Interface web disponible sur le port 8081.
airflow-scheduler	Up	Orchestrateur prêt à lancer les tâches.

Versions fixées par le Dockerfile fourni : Airflow 2.10.2, dbt-core 1.12.5 et dbt-snowflake 1.12.1. La validation a utilisé Python 3.12.6 dans le conteneur.

PowerShell - si le démarrage échoue

```
docker compose logs --tail 80 airflow-init airflow-scheduler
```

À retenir
Les dossiers logs et plugins peuvent être créés lors du montage. Le volume airflow-data conserve la base de métadonnées Airflow. Ne pas supprimer les volumes pour résoudre un simple problème d’identifiants.

GUIDE DE REPRODUCTION / 16

Tester la connexion par clé

Dans PowerShell, à la racine du projet, exécuter :

PowerShell

```
docker compose exec airflow-scheduler dbt debug --connection --project-dir `
    /opt/airflow/mon_projet --profiles-dir /home/airflow/.dbt
```

Les options indiquent explicitement le dossier du projet et celui du profil dans le conteneur. –connection teste la connexion sans le contrôle des outils de développement, dont Git.

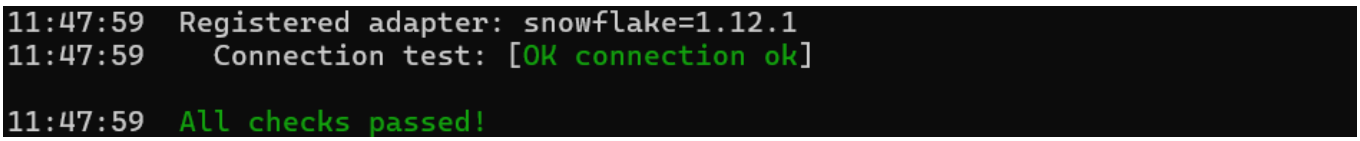


Figure 05. Preuve observée : Connection test [OK connection ok], puis All checks passed pour le test de connexion.

Contrôle	Ce que cela confirme
Profil trouvé et valide	Le YAML est lisible et le montage du dossier .dbt fonctionne.
Connexion réussie	Le compte, le login et la paire de clés permettent une session Snowflake.
Limite	Ce contrôle ne construit pas encore les modèles et ne valide pas la qualité des données.

Si le test échoue

• **390100** : vérifier account, login_name et la méthode d'authentification.

• **394508 / TOTP requis** : vérifier que password a été retiré et que private_key_path est présent.

• **Clé introuvable** : vérifier les deux Test-Path de la page 14 et le montage Compose.

• **JWT invalide** : vérifier que la clé publique associée correspond à la clé privée locale et que l'horloge Windows est à l'heure.

> **À retenir**
> Le message authenticator : None peut apparaître dans la sortie dbt même si la connexion par clé fonctionne. Le critère de validation est le résultat du test de connexion.

GUIDE DE REPRODUCTION / 17

Construire les modèles et lancer les tests

Depuis la racine du projet dans PowerShell :

PowerShell - construction

```
docker compose exec airflow-scheduler dbt run --select staging marts `
    --project-dir /opt/airflow/mon_projet --profiles-dir /home/airflow/.dbt
```

Résultat attendu : cinq modèles construits, soit quatre vues de staging et une table de résultat ; aucune erreur.

PowerShell - tests

```
docker compose exec airflow-scheduler dbt test --select staging marts `
    --project-dir /opt/airflow/mon_projet --profiles-dir /home/airflow/.dbt
```

Test du projet	Condition attendue
unique sur stg_customers.customer_id	Aucun identifiant client dupliqué.
accepted_values sur stg_orders.order_status	Valeurs Completed, Pending ou Cancelled.

Sélection des modèles
Conserver –select staging marts. Le dossier models/example contient un modèle de démonstration dbt avec un identifiant nul, dont le test not_null échouerait lors d'un test global. Les deux tests métier ne couvrent pas toutes les règles possibles.

CAPTURE À AJOUTER - C04
Construction et tests dbt réussis
Insérer la fin de dbt run avec PASS=5, puis dbt test avec PASS=2 et ERROR=0. Recadrer les invites et identifiants.

GUIDE DE REPRODUCTION / 18

Vérifier le résultat historique

Dans Snowflake, consulter la table créée par dbt :

SQL

```
SELECT *
FROM FINANCE_DB.RAW.DAILY_ORDER_REVENUE
ORDER BY ORDER_DATE DESC, ORDER_ID;
```

ORDER_ID	ORDER_DATE	TOTAL_PRICE
1	Date de l’insertion	200

Avant Kafka, une seule ligne doit être présente sur une base neuve. Le montant provient de 2 unités × 100. La date suit CURRENT_DATE() au moment de l’insertion.

Comprendre le modèle SQL du projet

models/marts/daily_order_revenue.sql

```
SELECT
    o.order_date,
    o.order_id,
    SUM(total_price) AS total_price
FROM {{ ref('stg_orders') }} o
LEFT JOIN {{ ref('stg_order_items') }} oi
    ON o.order_id = oi.order_id
GROUP BY 1, 2
```

Interpréter correctement les montants
Le résultat contient une ligne par date et par commande, malgré le nom daily_order_revenue. Il additionne les lignes de toutes les commandes, y compris Pending et Cancelled : il ne représente pas un chiffre d’affaires encaissé filtré sur Completed.

Le calcul est réalisé dans Snowflake. dbt résout ref(), ordonne les dépendances et envoie le SQL nécessaire à la création des objets. Les modèles de staging unissent les tables historiques et les tables de flux.

GUIDE DE REPRODUCTION / 19

Lancer le DAG dans Airflow

1. Ouvrir **http ://localhost :8081**. Le service airflow-init du projet crée un compte local admin / admin. Ces identifiants sont prévus pour cette démonstration locale.

2. Dans DAGs, ouvrir **dbt_snowflake_workflow**. Activer l'interrupteur, puis cliquer sur le triangle et Trigger DAG pour lancer une exécution immédiate.

3. Ouvrir Graph ou la grille de tâches. Attendre les quatre états success de la même exécution. Une colonne correspond à une exécution : ne pas mélanger les états de colonnes différentes.

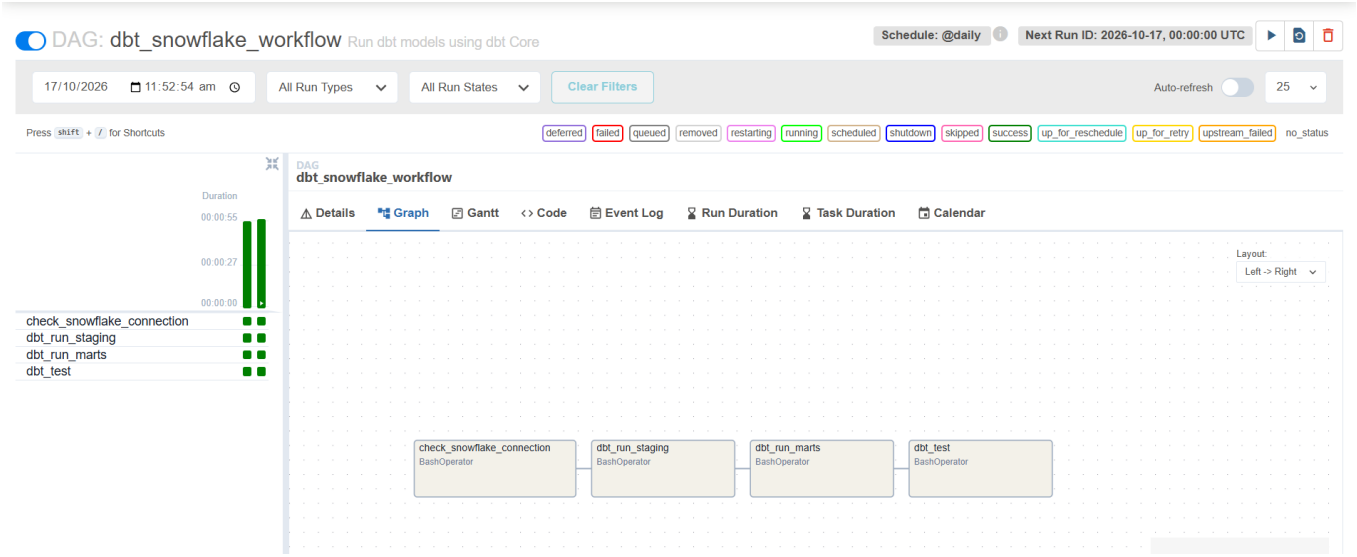


Figure 06. Preuve observée : les quatre tâches sont vertes sur deux exécutions. Les cases de la grille à gauche permettent de vérifier le succès.

Tâche	Action
check_snowflake_connection	dbt debug –connection
dbt_run_staging	dbt run –select staging
dbt_run_marts	dbt run –select marts
dbt_test	dbt test –select staging marts

Le DAG est planifié en @daily avec catchup=False. La première activation peut également créer une exécution planifiée ; deux exécutions visibles ne signifient pas nécessairement deux clics manuels. À chaque échec, le DAG prévoit une nouvelle tentative après cinq minutes.

GUIDE DE REPRODUCTION / 20

Démarrer Kafka et créer le topic

La suite se déroule dans le sous-dossier kafka-lab. Ce fichier Compose démarre un broker Kafka 3.8.0 en mode KRaft et Kafka UI ; aucun ZooKeeper n’est requis par cette configuration.

PowerShell

```
$Projet = "C:\Projets\pipeline-ventes"
Set-Location "$Projet\kafka-lab"
docker compose up -d
docker compose ps
```

Créer le topic explicitement, car la création automatique est désactivée dans le fichier Compose. Attendre que le broker soit prêt ; en cas de refus de connexion initial, vérifier ses logs puis relancer.

PowerShell

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server `
    localhost:9092 --create --if-not-exists --topic orders --partitions 1 `
    --replication-factor 1
```

PowerShell - vérifier le topic

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server `
    localhost:9092 --list
```

La liste doit contenir orders. Les scripts Python Windows utilisent localhost :9092 ; Kafka UI utilise kafka :19092 à l’intérieur du réseau Docker de cette stack.

PowerShell - diagnostic du broker

```
docker compose logs --tail 80 kafka
```

GUIDE DE REPRODUCTION / 21

Adapter le consommateur à la clé RSA

Le script original consumer_snowflake.py lit cfg["password"]. Le profil par clé n'a plus ce champ : il faut modifier la fonction de connexion avant le lancement.

PowerShell

```
Set-Location "$Projet\kafka-lab"
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
notepad .\consumer_snowflake.py
```

Remplacer uniquement la fonction snowflake_connect(), jusqu'à la ligne précédant def flush(...), par le bloc suivant. Conserver les imports existants os, yaml et snowflake.connector. [R5]

Code Python à enregistrer dans consumer_snowflake.py

```
def snowflake_connect():
    with open(
        os.path.expanduser("~/.dbt/profiles.yml"),
        encoding="utf-8-sig"
    ) as f:
        profile = yaml.safe_load(f)["mon_projet"]

    cfg = profile["outputs"][profile["target"]]

    return snowflake.connector.connect(
        account=cfg["account"],
        user=cfg["user"],
        private_key_file=os.path.expanduser(
            "~/.dbt/snowflake_key.p8"
        ),
        role=cfg["role"],
        warehouse=cfg["warehouse"],
        database=cfg["database"],
        schema=cfg["schema"],
    )
```

Chemin de la clé
Le consommateur lit le même profil que dbt, mais reconstruit le chemin local Windows de la clé. Il ne réutilise pas directement private_key_path, qui pointe vers le conteneur Linux.

Le consommateur crée les tables *_STREAM si elles n'existent pas, écrit commandes et lignes dans une transaction, puis valide sa position Kafka après le chargement. Cela permet des reprises avec livraison au moins une fois ; dbt déduplique le flux par identifiant.

GUIDE DE REPRODUCTION / 22

Lancer les deux terminaux Python

Pour le jeu minimal à un client et un produit, ouvrir producer.py et remplacer les deux tirages suivants dans build_order(). Cela évite des références à des clients ou produits absents.

Deux remplacements dans producer.py ; conserver le reste du code

```
# Ancien : random.randint(1, 10) pour product_id
"product_id": 1

# Ancien : random.randint(1, 100) pour customer_id
"customer_id": 1
```

Les clés ci-dessus se trouvent dans des dictionnaires existants : remplacer seulement la valeur numérique, en conservant les virgules et l'indentation autour. Les prix et les quantités peuvent rester aléatoires.

Terminal A : consommateur

PowerShell - dans kafka-lab

```
.\.venv\Scripts\python.exe consumer_snowflake.py
```

Attendre le message indiquant que la connexion Snowflake et les tables sont prêtes. Laisser ce terminal ouvert.

Terminal B : producteur

Nouveau PowerShell

```
$Projet = "C:\Projets\pipeline-ventes"
Set-Location "$Projet\kafka-lab"
.\.venv\Scripts\python.exe producer.py --count 20 --start-id 10001
```

Le producteur envoie vingt commandes, une par seconde par défaut, puis s'arrête. Le consommateur reste actif. Attendre sa confirmation de chargement : avec vingt messages, l'envoi se produit normalement environ trente secondes après le premier message reçu.

GUIDE DE REPRODUCTION / 23

Inspecter Kafka et les données brutes

Ouvrir **http ://localhost :8085**. Dans Kafka UI, ouvrir le cluster kafka-lab, puis Topics → orders. Consulter les messages et vérifier leurs champs.

Exemple pédagogique de message ; les valeurs réelles varient

```
{
  "order_id": 10001,
  "customer_id": 1,
  "order_date": "2026-10-17",
  "status": "Completed",
  "total_amount": 200,
  "items": [
    {"product_id": 1, "quantity": 2, "unit_price": 100}
  ]
}
```

SQL

```
SELECT COUNT(*) AS NB_LIGNES_BRUTES,
       COUNT(DISTINCT ID) AS NB_COMMANDES_DISTINCTES
FROM FINANCE_DB.RAW.ORDERS_STREAM;

SELECT ID, STATUS, KAFKA_PARTITION, KAFKA_OFFSET, INGESTED_AT
FROM FINANCE_DB.RAW.ORDERS_STREAM
ORDER BY INGESTED_AT DESC, ID
LIMIT 20;
```

Pour le premier envoi sur des tables vides, vingt lignes brutes et vingt identifiants distincts sont attendus. Après une reprise ou une relecture du topic, le nombre brut peut être plus élevé ; vérifier aussi le résultat dédupliqué.

GUIDE DE REPRODUCTION / 24

Valider le résultat de bout en bout

Après la confirmation du micro-lot, relancer une seule fois le DAG Airflow avec Trigger DAG. Attendre le succès des quatre tâches, puis exécuter dans Snowflake :

SQL

```
SELECT *
FROM FINANCE_DB.RAW.DAILY_ORDER_REVENUE
ORDER BY ORDER_DATE DESC, ORDER_ID;

SELECT SOURCE_SYSTEME, COUNT(*) AS NB_COMMANDES
FROM FINANCE_DB.RAW.STG_ORDERS
GROUP BY SOURCE_SYSTEME;
```



Figure 07. Résultat réellement observé : 21 lignes dans DAILY_ORDER_REVENUE, dont la commande 1 à 200 et les identifiants Kafka 10001 à 10020.

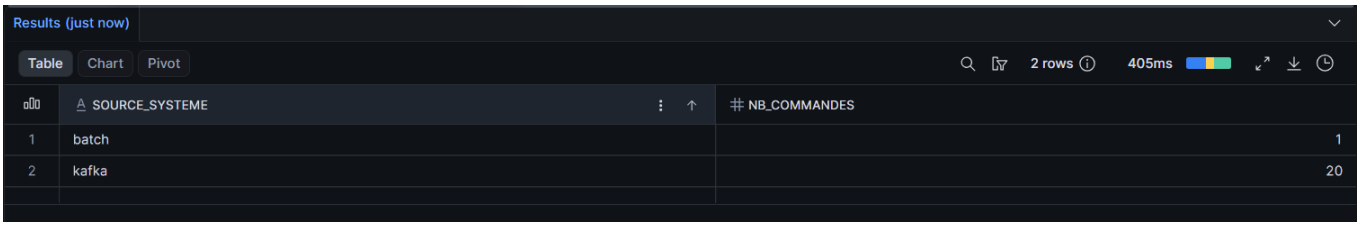


Figure 08. Répartition réellement observée : 1 commande batch et 20 commandes kafka.

> **À retenir**
> Résultat attendu sur une base neuve : 21 commandes. Les captures proviennent de la session réalisée le 17 octobre 2026 ; la date et les montants aléatoires de votre propre exécution peuvent différer.

GUIDE DE REPRODUCTION / 25

Rejouer la démonstration

Conserver le consommateur actif et choisir une nouvelle plage d’identifiants à chaque nouvel envoi. Réutiliser 10001 peut mettre à jour la version retenue par dbt au lieu d’augmenter le nombre de commandes.

PowerShell - dans kafka-lab

```
.\.venv\Scripts\python.exe producer.py --count 20 --start-id 20001
```

Attendre le chargement dans le terminal du consommateur. Relancer le DAG Airflow, puis contrôler la répartition par source et la table finale. Après ce deuxième envoi, 41 commandes sont attendues si aucune autre donnée n’a été ajoutée.

Scénario de présentation en cinq actions

Action	Preuve à montrer
1. Montrer l’état initial	La table résultat et sa quantité de commandes.
2. Lancer le producteur	Le terminal et les confirmations de livraison.
3. Montrer l’arrivée du flux	Kafka UI puis ORDERS_STREAM dans Snowflake.
4. Déclencher Airflow	Les quatre tâches qui passent au vert.
5. Montrer le résultat actualisé	DAILY_ORDER_REVENUE et le comptage par source.

Contrôler la granularité et les doublons

SQL

```
SELECT ORDER_ID, COUNT(*) AS NB
FROM FINANCE_DB.RAW.STG_ORDERS
GROUP BY ORDER_ID
HAVING COUNT(*) > 1;

SELECT COUNT(*) AS NB_COMMANDES,
       SUM(TOTAL_PRICE) AS MONTANT_TOTAL_COMMANDES
FROM FINANCE_DB.RAW.DAILY_ORDER_REVENUE;
```

La première requête doit renvoyer zéro ligne sur la démonstration. Les identifiants de flux doivent également être distincts de ceux de l’historique : l’union batch / flux ne déduplique pas les conflits entre ces deux origines.

GUIDE DE REPRODUCTION / 26

Arrêter et redémarrer sans perdre les données

Arrêt après la démonstration

Dans le terminal du consommateur, appuyer sur Ctrl + C et attendre sa fin. Le script tente de vider le dernier lot avant de quitter. Vérifier le message de chargement avant d'arrêter les conteneurs.

PowerShell

```
$Projet = "C:\Projets\pipeline-ventes"
Set-Location "$Projet\kafka-lab"
docker compose stop
Set-Location $Projet
docker compose stop
```

stop conserve les conteneurs et les volumes. Les données Snowflake restent dans le compte cloud. Éviter docker compose down -v : cette option supprime les volumes locaux concernés.

Redémarrage

PowerShell

```
$Projet = "C:\Projets\pipeline-ventes"
Set-Location $Projet
docker compose up -d
Set-Location "$Projet\kafka-lab"
docker compose up -d
.\.venv\Scripts\python.exe consumer_snowflake.py
```

Ouvrir un second terminal pour le producteur et utiliser une nouvelle plage d'identifiants. La création du topic est inutile s'il existe déjà et si le volume Kafka a été conservé.

> **À retenir**
> Airflow ne planifie rien lorsque ses conteneurs sont arrêtés. Avec catchup=False, il ne rejoue pas automatiquement toutes les périodes manquées. Le consommateur ne tourne pas dans Docker : il faut le relancer séparément sur Windows.

Vérifications rapides

PowerShell

```
# Racine : Airflow
Set-Location $Projet
docker compose ps
# Sous-dossier : Kafka
Set-Location "$Projet\kafka-lab"
docker compose ps
```

GUIDE DE REPRODUCTION / 27

Dépannage : Snowflake et authentification

Symptôme	Cause probable et correction
Cannot perform CREATE TABLE ... no current database	Exécuter USE DATABASE FINANCE_DB ; puis USE SCHEMA RAW ; dans la session. Ou utiliser FINANCE_DB.RAW.NOM_TABLE.
No active warehouse selected	Exécuter USE WAREHOUSE FINANCE_WH ; et vérifier les droits USAGE / calcul du rôle.
Object does not exist or not authorized	Vérifier la base, le schéma, le nom exact et le rôle. Créer les six tables attendues avant dbt.
390100 : incorrect username or password	Vérifier account, puis login_name avec SHOW USERS. CURRENT_USER() donne le nom d'objet, pas nécessairement le login.
394508 : MFA with TOTP is required	Utiliser le profil par clé des pages 12 à 14. Ne pas désactiver la MFA pour résoudre une connexion automatisée.
JWT / private key invalid	Vérifier la correspondance des clés et le chemin local / conteneur. Réutiliser la même clé privée que celle dont la clé publique a été associée.
KeyError : password dans le consommateur	Le script est encore dans sa version initiale. Remplacer sa fonction snowflake_connect() par celle de la page 21.
FileNotFoundError pour la clé	dbt Docker utilise /home/airflow/.dbt/... ; le consommateur Windows utilise ~/.dbt/.... Vérifier le montage et le nom du fichier.

Obtenir un diagnostic sans secret

SQL

```
SHOW USERS LIKE 'VOTRE_NOM_OBJET_UTILISATEUR';
SELECT "name", "login_name", "has_rsa_public_key"
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

has_rsa_public_key confirme qu'une clé est enregistrée, pas qu'elle correspond à votre fichier local. Pour un partage de diagnostic, transmettre le message d'erreur et masquer les identifiants. Ne jamais joindre le fichier snowflake_key.p8.

GUIDE DE REPRODUCTION / 28

Dépannage : Windows, Docker et Kafka

Symptôme	Action
Copy-Item : fichier introuvable	Revenir à la racine avec Set-Location $Projet. Vérifier Get-Item .\profiles.example.yml avant la copie.
profiles.yml introuvable / YAML invalide	Vérifier le nom sans .txt, les espaces d’indentation et les clés mon_projet / outputs / dev.
Git ERROR lors de dbt debug	Le contrôle Git est séparé de la connexion. Utiliser dbt debug –connection pour le diagnostic du projet sans packages Git. Installer Git dans l’image si nécessaire, pas seulement sur Windows.
Port déjà utilisé	Identifier le service existant avant de l’arrêter. Adapter le port hôte du Compose si nécessaire ; les interfaces du guide utilisent 8081 et 8085.
Kafka : connexion refusée	Vérifier docker compose ps puis logs kafka dans kafka-lab. Attendre le démarrage du broker et relancer la création du topic.
Aucun lot chargé après 20 messages	Laisser le consommateur ouvert au moins 30 secondes après le premier message ; consulter son erreur éventuelle. Vérifier aussi le groupe et les offsets déjà validés.
Table finale inchangée après Kafka	Vérifier ORDERS_STREAM, puis déclencher le DAG Airflow. Le chargement Kafka ne lance pas dbt automatiquement.
Tâche Airflow rouge	Cliquer sur le carré de la tâche puis Logs. Corriger la cause et relancer une exécution. Ne pas multiplier les déclenchements pendant un échec.

Commandes de diagnostic

PowerShell

```
# Depuis la racine du projet
docker compose logs --tail 80 airflow-scheduler
docker compose exec airflow-scheduler airflow dags list-import-errors
# Depuis kafka-lab
docker compose logs --tail 80 kafka
```

Si Git est requis : ajouter une installation apt-get de git sous USER root dans le Dockerfile, revenir ensuite à USER airflow, puis reconstruire avec docker compose up -d –build. [R6]

GUIDE DE REPRODUCTION / 29

Limites et portée des résultats

Le guide valide un projet pédagogique fonctionnel sur un petit volume. Les affirmations ci-dessous permettent de le présenter avec précision.

Point	Ce que fait réellement le projet
Fréquence	Kafka et le consommateur fonctionnent en continu ; dbt reconstruit la table au lancement manuel ou quotidien du DAG.
Granularité	DAILY_ORDER_REVENUE regroupe par date et identifiant de commande. Pour un total journalier, agréger cette table par ORDER_DATE.
Statuts	Les montants Pending et Cancelled sont inclus. Un indicateur de ventes finalisées doit filtrer Completed.
Tests	Deux règles métier seulement : unicité client et statuts autorisés. Ajouter des tests not_null, relations et cohérence des montants si nécessaire.
Livraison Kafka	Chargement par micro-lots avec validation de position après écriture. Les reprises peuvent produire des doublons bruts ; dbt déduplique le flux par identifiant.
Catalogue minimal	Le guide restreint le producteur au client 1 / produit 1. Le producteur original attend un catalogue plus large, qui n’est pas livré en CSV.
Infrastructure	Un seul broker, réplication 1 et Airflow SQLite / SequentialExecutor. Cette configuration ne démontre pas une haute disponibilité en production.
Comptes et clés	ACCOUNTADMIN et la clé non chiffrée reprennent une démonstration locale. Pour un déploiement partagé, prévoir des rôles dédiés et une gestion adaptée des secrets.

Exemple : total des commandes par jour

SQL

```sql
SELECT ORDER_DATE,
       SUM(TOTAL_PRICE) AS MONTANT_TOTAL_COMMANDES
FROM FINANCE_DB.RAW.DAILY_ORDER_REVENUE
GROUP BY ORDER_DATE
ORDER BY ORDER_DATE;
```

Le terme « montant total des commandes » décrit ici le calcul sans supposer qu’il s’agit d’un revenu encaissé. Ne pas annoncer de mesures de performance ou de volumes qui n’ont pas été testés.

GUIDE DE REPRODUCTION / 30

Checklist et captures à compléter

Avant de transmettre le projet à une autre personne, vérifier les points suivants :

Contrôle final	Preuve attendue
Environnement	Docker et Python accessibles ; racine du projet identifiable.
Snowflake	Six tables brutes présentes et données de démonstration chargées.
Connexion	dbt debug –connection réussi avec une clé locale.
Transformation	Cinq modèles construits et deux tests réussis.
Orchestration	Les quatre tâches d’une même exécution Airflow sont vertes.
Streaming	Vingt nouvelles commandes reçues dans Snowflake.
Résultat final	Vingt et une commandes, réparties en batch = 1 et kafka = 20.

Références officielles

[R1] Docker Desktop sur Windows

[R2] Snowflake Workspaces

[R3] Snowflake : authentification par clés

[R4] dbt : connexion Snowflake

[R5] Connecteur Python Snowflake

[R6] Airflow : personnaliser une image Docker

Sources du guide : code de l’archive fournie et captures de la session de validation. Projet attribué dans son README à Hajar Oussama et Zouhair Serrar, sous l’encadrement du Prof. Karim. Guide de reproduction préparé pour Zouhair Serrar. Documentation consultée le 17 octobre 2026.